Python语言与系统设计

（课程概述）

Python Language & System Design

第1章 Python课程概述



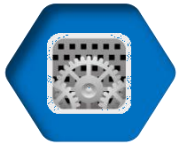
合肥工业大学 计算机与信息学院

第1章　概 述


1.1 引言


1.2 Python的安装与启动


1.3 Python基础知识


1.4 Python代码规范


1.5 Python快速入门

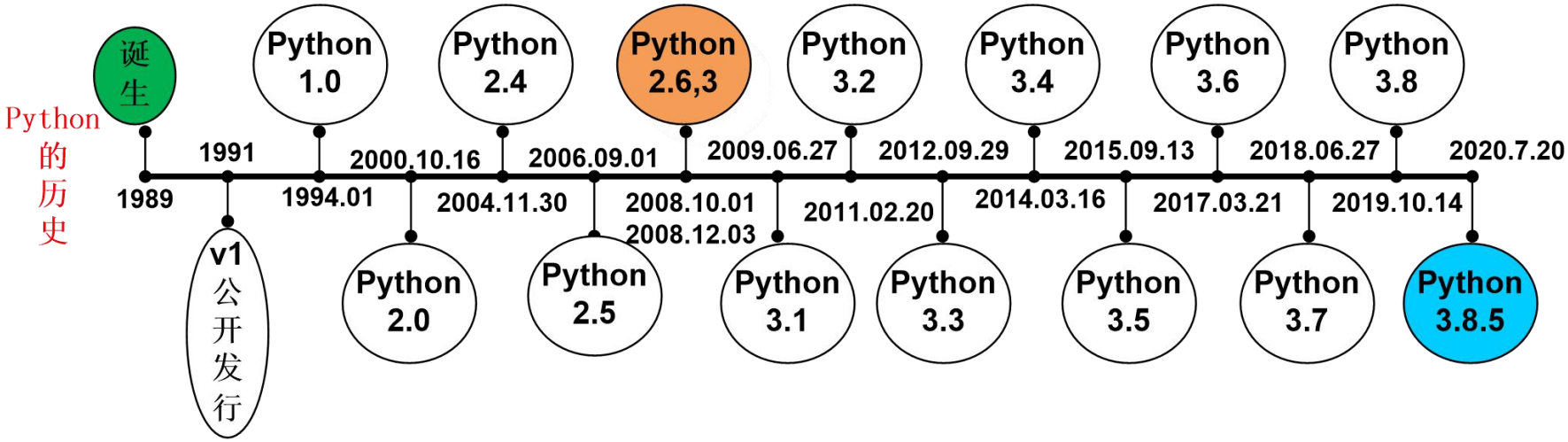

1.6 本章小结

1.1 引言

- Guido van Rossum：Python语言创立者 1989

命名：Monty Python's Flying Circus





https://www.python.org/doc/versions/



1.1 引言

Python语言的魅力

- Python是一门跨平台、开源、免费的解释型高级动态编程语言
  - ✓ 支持伪编译将Python源程序转换为**字节码**来优化程序和提高运行速度
  - ✓ 支持使用py2exe、pyinstaller或cx_Freeze工具将Python程序**转换为二进制可执行文件**。
- Python支持命令式编程（How to do）、函数式编程（What to do），完全支持**面向对象程序设计**
  - ✓ 语法简洁清晰，**拥有大量的几乎支持所有领域应用开发的成熟扩展库**。
- **胶水语言**：可以把多种不同语言编写的程序融合到一起实现无缝拼接，更好地发挥不同语言和工具的优势，满足不同应用领域的需求。
  - ✓ **Web and Internet Development: Django、Pyramid、Flask**
  - ✓ **Scientific and Numeric: NumPy、SciPy、Matplotlib**
  - ✓ **Desktop GUIs: PyQT，WXPython，TkInter**
  - ✓ **Software Development**
  - ✓ **Business Applications**
  - ✓ **Education**
  - ✓ ……



1.1 引言

- 课程概况：

开课单位	计算机与信息学院	课程类型	专业选修课程
课程名称	Python语言与系统设计	课程代码	0521760X
开课学期	第三学期	学时/学分	(24理论+8实践) /2
选课对象	主要面向计算机科学与技术专业		

- 课程教材：

董付国. 《Python程序设计》(第2/第3版). 清华大学出版社

- 评估方式 ：

期末考试	40%
课堂测试	20%
实验分析报告	20%
设计报告	20%



1.1 引言

课次	学时	教 学 主 要 内 容
1	2	第一章：Python基础知识
2	4	第二章：Python序列与复杂数据结构
3	2	第三章：程序的控制结构
4	2	第四章：字符串与正则表达式
5	2	第五章：函数设计与使用
6	4	第六章：面向对象程序设计
7	2	第七章：文件操作与异常处理
8	2	第八章：科学计算与可视化（选学）
9	2	第九章：GUI编程（选学）
10	2	第十一章：多线程与多进程编程（选学）
11	2	实验一：Python序列与复杂数据结构
12	2	实验二：判断、循环、函数、文件
13	4	实验三：综合设计实验：面向实际应用问题编程设计算法

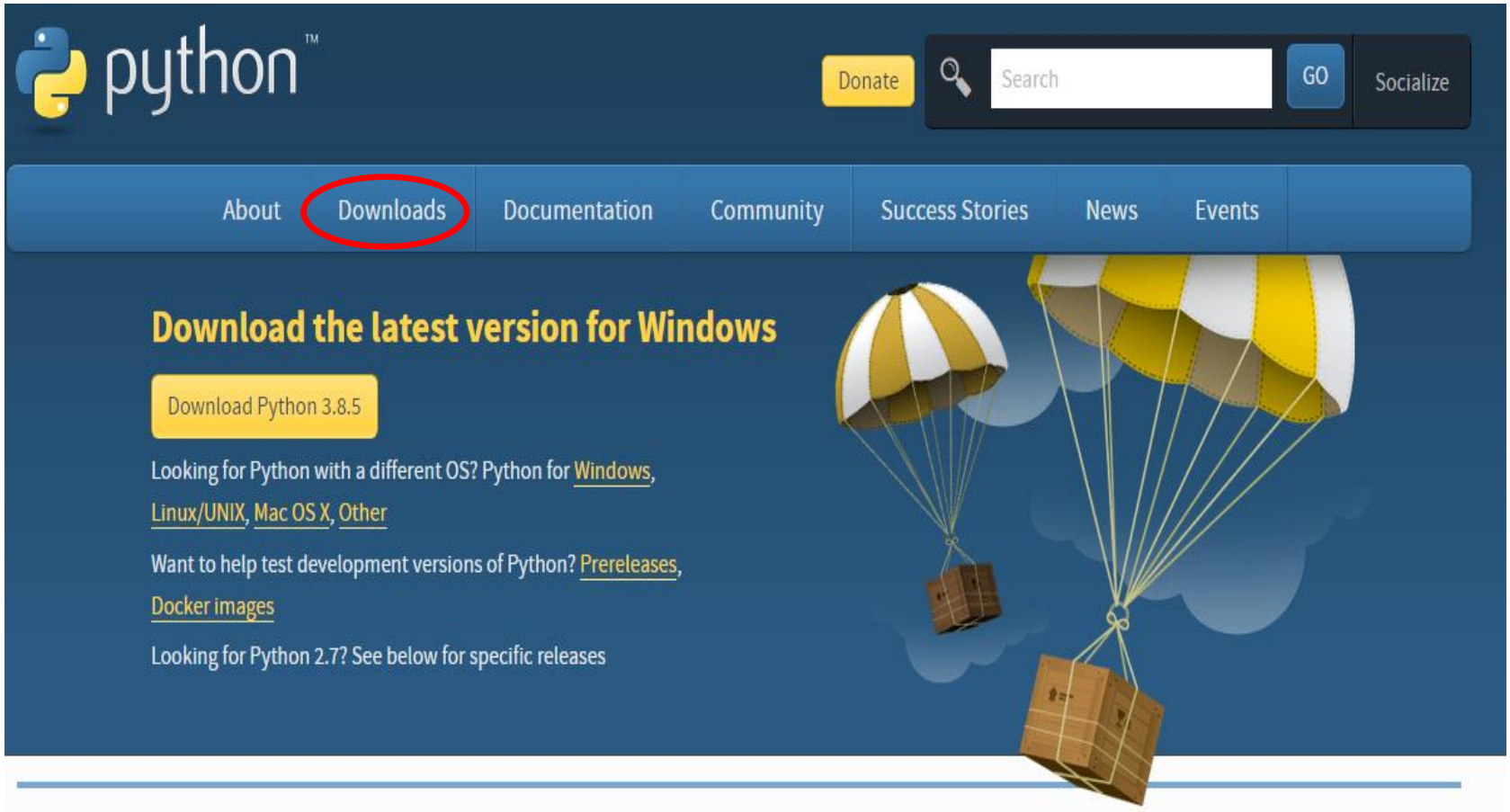
1.2 Python的安装与启动

- 默认编程环境：**IDLE** (**Integrated Development & Learning Environment**)
- 其他常用开发环境
  - **pyCharm**
  - **Anaconda3**
  - **Eclipse+PyDev**
  - wingIDE
  - Eric
  - PythonWin
  - zwPython
- 几个重要网址
  - https://www.python.org/
  - https://www.python.org/doc/
  - http://bugs.python.org/
  - https://hackerone.com/python
  - http://stackoverflow.com/questions/tagged/python

1.2 Python的安装与使用

➢1.2.1 Python的安装

www.python.org/downloads/



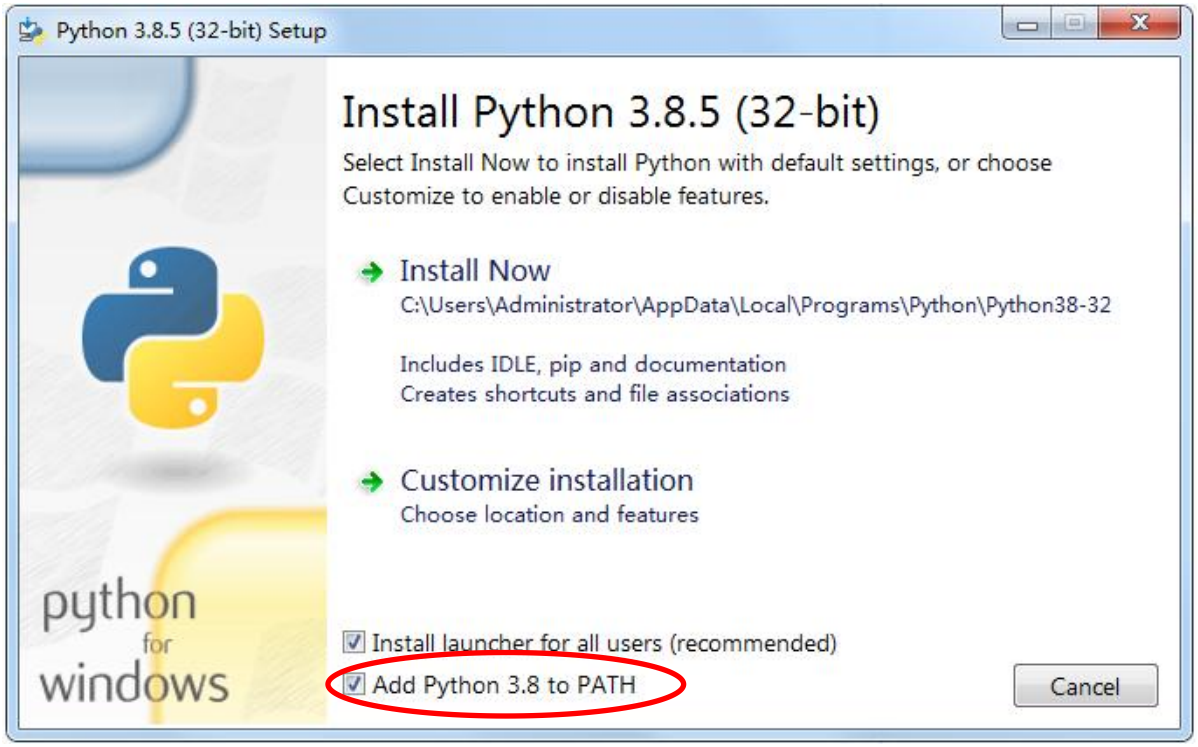
1.2 Python的安装与启动

➢1.2.1 Python的安装

- 高级别的3.0系列不兼容2.0系列

- Python 3.x系列已经成为主流

安装时：
Add python.exe to Path
需要勾选



- 多版本共存与切换简便方法：修改系统环境变量path

1.2 Python的安装与启动

1.2.1 Python的安装

使用pip管理第三方包

- pip工具常用命令

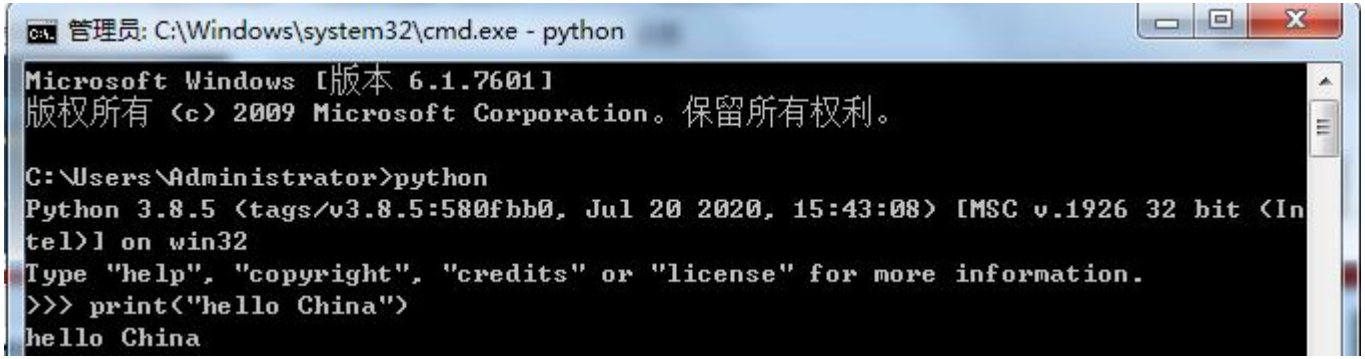
pip命令示例	说明
pip download SomePackage[==version]	下载扩展库的指定版本，不安装
pip freeze [> requirements.txt]	以requirements的格式列出已安装模块
pip list	列出当前已安装的所有模块
pip install SomePackage[==version]	在线安装SomePackage模块的指定版本
pip install SomePackage.whl	通过whl文件离线安装扩展库
pip install package1 package2 ...	依次（在线）安装package1、package2等扩展模块
pip install -r requirements.txt	安装requirements.txt文件中指定的扩展库
pip install --upgrade SomePackage	升级SomePackage模块
pip uninstall SomePackage[==version]	卸载SomePackage模块的指定版本

注：slides参考：董付国. 《Python程序设计》(第2版). 清华大学出版社, 2018.

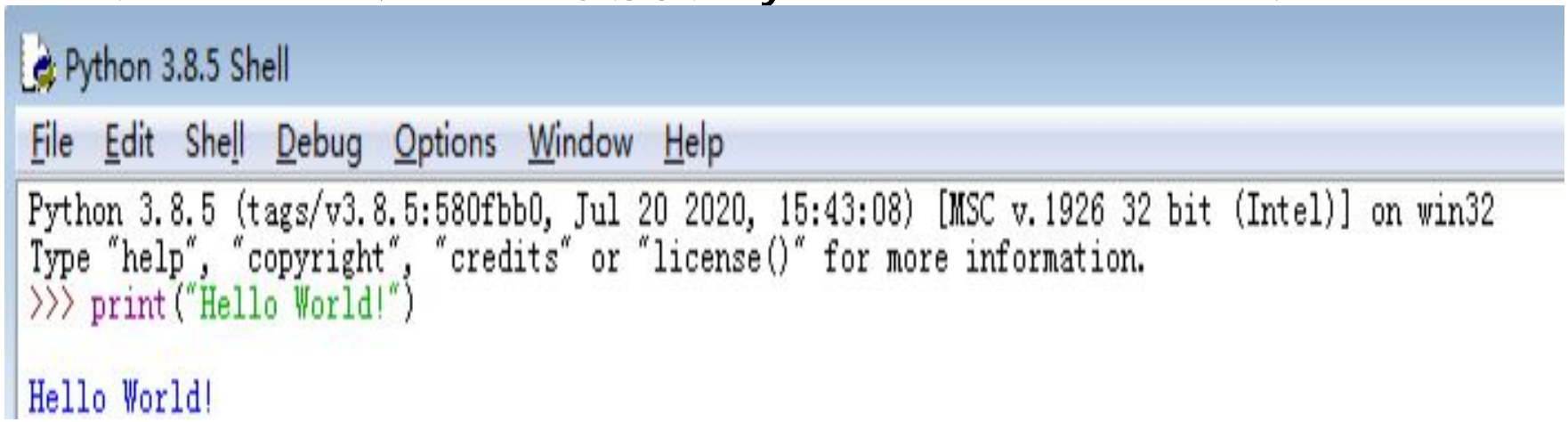
1.2 Python的安装与启动

1.2.2 Python的启动

- 方法1：启动Windows命令行工具，输入python

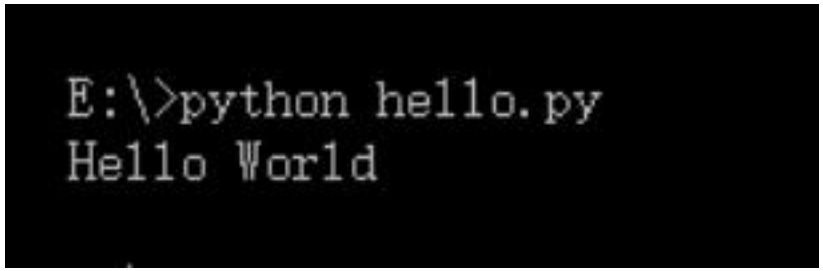


- 方法2：调用IDLE来启动Python图形化运行环境



1.2.2 Python的启动

- 方法3：按照语法格式编写代码，编写可以用任何文本编辑器，保存为文件(.py或.pyw文件)



- 方法4：打开IDLE，点击Ctrl+N打开一个新窗口，输入语句并保存(.py或.pyw文件)，使用快键建F5（Run Module）即可运行该程序

1.2.2 Python的启动

- 方法5：将Python集成到Eclipse、PyCharm等面向较大规模项目开发的集成开发环境中

  采用上述某个方法，执行：

```
>>> print("Hello World")
Hello World
>>> print("世界，你好")
世界，你好
>>>
```

1.2 Python的安装与启动

IDLE环境下的常用的快捷键

- 除了撤销（Ctrl+Z）、全选（Ctrl+A）、复制（Ctrl+C）、粘贴（Ctrl+V）、剪切（Ctrl+X）等常规快捷键外，其他比较常用的快捷键如下表所示。

快捷键	功能说明
Alt+p	浏览历史命令（上一条）
Alt+n	浏览历史命令（下一条）
Ctrl+F6	重启Shell，之前定义的对象和导入的模块全部失效
F1	打开Python帮助文档
Alt+/	自动补全前面曾经出现过的单词，如果之前有多个单词具有相同前缀，则在多个单词中循环选择
Ctrl+]	缩进代码块
Ctrl+[	取消代码块缩进
Alt+3	注释代码块
Alt+4	取消代码块注释。
Tab	补全单词

1.3 基础知识---Python对象模型

1.3.1 Python对象模型

- **对象是python中最基本的概念**，在python中处理的一切都是对象。
- 包含许多内置对象可直接使用，如数字、字符串、列表、del等；
- 非内置对象需要导入模块才能使用，如正弦函数sin(x)，随机数产生函数random( )等。

对象类型	类型名称	示例	简要说明
数字	int, float, complex	1234, 3.14, 1.3e5, 3+4j	数字大小没有限制，内置支持复数及其运算
字符串	str	'swfu', "I'm student", '"Python "', r'abc', R'bcd'	使用单引号、双引号、三引号作为定界符，以字母r或R引导的表示原始字符串
字节串	bytes	b'hello world'	以字母b引导，可以使用单引号、双引号、三引号作为定界符
列表	list	[1, 2, 3]，['a', 'b', ['c', 2]]	所有元素放在一对方括号中，元素之间使用逗号分隔，其中的元素可以是任意类型
字典	dict	{1:'food' ,2:'taste', 3:'import'}	所有元素放在一对大括号中，元素之间使用逗号分隔，元素形式为“键:值”
元组	tuple	(2, -5, 6), (3,)	不可变，所有元素放在一对圆括号中，元素之间使用逗号分隔，如果元组中只有一个元素的话，后面的逗号不能省略
集合	set frozenset	{'a', 'b', 'c'}	所有元素放在一对大括号中，元素之间使用逗号分隔，元素不允许重复;另外，set是可变的，而frozenset是不可变的

1.3 基础知识---Python对象模型

1.3.1 Python对象模型

对象类型	类型名称	示例	简要说明
布尔型	bool	True, False	逻辑值，关系运算符、成员测试运算符、同一性测试运算符组成的表达式的值一般为True或False
空类型	NoneType	None	空值
异常	Exception、ValueError、TypeError		Python内置大量异常类，分别对应不同类型的异常
文件		f = open('data.dat', 'rb')	open是Python内置函数，使用指定的模式打开文件，返回文件对象
其他可迭代对象		生成器对象、range对象、zip对象enumerate对象、map对象、filter对象等等	具有惰性求值的特点，除range对象之外，其他对象中的元素只能看一次
编程单元		函数（使用def定义） 类（使用class定义） 模块（类型为module）	类和函数都属于可调用对象，模块用来集中存放函数、类、常量或其他对象

1.3 基础知识---Python变量

1.3.2 Python变量

- 在Python中，不需事先声明变量名及其类型，直接赋值即可创建各种类型的对象变量。这一点适用于Python任意类型的对象。

✓ 例如：

```
>>> x = 3
```

凭空出现一个整型变量x

创建了整型变量x，并赋值为3。

```
>>> x = 'Hello world.'
```

新的字符串变量，再也不是原来的x了

创建了字符串变量x，并赋值为'Hello world.'。

- Python属于强类型编程语言，Python解释器会根据赋值或运算来自动推断变量类型。Python还是一种动态类型语言，变量的类型也是可以随时变化的。

✓ 例如：

```
>>> x = 3
>>> print(type(x))          #type():查看变量类型
<class 'int'>
>>> x = 'Hello world.'
>>> print(type(x))
<class 'str'>
>>> isinstance(3, int)      #isinstance():测试对象是否是某个类型的实例
True
```

1.3 基础知识---Python变量

- 如果变量出现在赋值运算符或复合赋值运算符（例如+=、*=等等）的左边则表示创建变量或修改变量的值，否则表示引用该变量的值
  - ✓ 例如：

```
>>> x = 3          #创建整型变量
>>> print(x**2)
9
>>> x += 6         #修改变量值
>>> print(x)
9
```

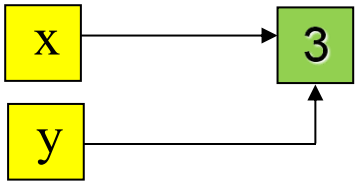
- 字符串和元组属于不可变序列，不能通过下标的方式来修改其中的元素值，试图修改元组中元素的值时会抛出异常。
  - ✓ 例如：

```
>>> x = (1,2,3)
>>> x[1] = 5
Traceback (most recent call last):
   File "<pyshell#7>", line 1, in <module>
      x[1] = 5
TypeError: 'tuple' object does not support item assignment
```

1.3 基础知识---Python变量

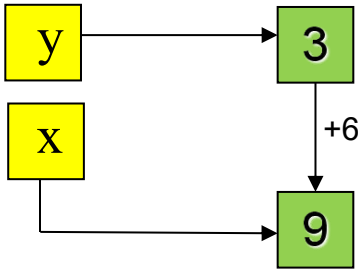
- 允许多个变量指向同一个值
  - ✓例如：

```
>>> x = 3
>>> id(x)
1786684560
>>> y = x
>>> id(y)
1786684560
```



- 当修改其中一个变量值后，其内存地址将会变化，但并不影响另一变量
  - ✓例如：

```
>>> x += 6
>>> id(x)
1786684752
>>> y
3
>>> id(y)
1786684560
```



1.3 基础知识---Python变量

- Python采用的是基于值的内存管理方式，如果为不同变量赋值为相同值，这个值在内存中只有一份，多个变量指向同一块内存地址。

✓ 例如：

```
>>> x = 3
>>> id(x)
10417624
>>> y = 3
>>> id(y)
10417624
>>> x = [1, 1, 1, 1]
>>> id(x[0]) == id(x[1])
True
```

1.3 基础知识---Python变量

- ✓ 变量名必须以字母或下划线开头，但以下划线开头的变量在Python中有特殊含义；
- ✓ 变量名中不能有空格以及标点符号（括号、引号、逗号、斜线、反斜线、冒号、句号、问号等）；
- ✓ 不能使用关键字作变量名，可以导入keyword模块后使用print(keyword.kwlist)查看所有Python关键字(33个)；

```
>>> import keyword
>>> print(keyword.kwlist)
['False', 'None', 'True', 'and', 'as', 'assert', 'break', 'class', 'continue',
def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if',
 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'retu
rn', 'try', 'while', 'with', 'yield']
```

- ✓ 不建议使用系统内置的模块名、类型名或函数名以及已导入的模块名及其成员名作变量名，这将会改变其类型和含义，可以通过dir(__builtins__)查看所有内置模块、类型和函数；
- ✓ 变量名对英文字母的大小写敏感，例如student和Student是不同的变量。

1.3.2 Python变量

- **Python中的变量并不直接存储值，而是存储了值的内存地址或者引用，这也是变量类型随时可以改变的原因。**
  - ✓ 赋值语句的执行过程是：首先把等号右侧表达式的值计算出来，然后在内存中寻找一个位置把值存放进去，最后创建变量并指向这个内存地址。

```
>>>x = 2**3
```

- Python具有**自动内存管理功能**，对于没有任何变量指向的值，Python自动将其删除。Python会跟踪所有的值，并自动删除不再有变量指向的值。
- 显式**使用del命令删除不需要的值或显式关闭不再需要访问的资源**，仍是一个好的习惯，同时也是一个优秀程序员的基本素养之一。

1.3.3 Python的数字类型

- 数字是不可变对象，可以表示任意大小的数字。
- ✓ 例如：

```
>>> a=99999999999999999999999999999999
>>> a*a
99999999999999999999999999999998000000000000000000000000000001
>>> a**3
999999999999999999999999999999970000000000000000000000000002999999999999999999999999999
```

- Python的IDLE交互界面可以当做简便计算器来使用。
- ✓ 例如：

```
>>> ((3**2) + (4**2)) ** 0.5
5.0
```

1.3 基础知识---Python的数字类型

■ 三类：整数类型、浮点类型与复数类型

① Python中的整数类型

- 十进制整数如，0、-1、9、123
- 十六进制整数，需要16个数字0、1、2、3、4、5、6、7、8、9、a、b、c、d、e、f来表示整数，必须以0x开头，如0x10、0xfa、0xabcdef
- 八进制整数，只需要8个数字0、1、2、3、4、5、6、7来表示整数，必须以0o开头，如0o35、0o11
- 二进制整数，只需要2个数字0、1来表示整数，必须以0b开头如，0b101、0b100

② 浮点数类型：浮点数又称小数

✓ 15.0、0.37、-11.2、1.2e2、314.15e-2



1.3 基础知识---Python的数字类型

③ Python内置支持复数类型

✓例如：

```
>>> a = 3+4j
>>> b = 5+6j
>>> c = a+b
>>> c
(8+10j)
>>> c.real          #查看复数实部
8.0
>>> c.imag          #查看复数虚部
10.0
>>> a.conjugate() #返回共轭复数
(3-4j)
>>> a*b             #复数乘法
(-9+38j)
>>> a/b             #复数除法
(0.6393442622950819+0.03278688524590165j)
```

1.3 基础知识---Python的数字类型

- Python 3.6.x开始支持在数字中间位置使用单个下划线作为分隔来提高数字的可读性，类似于数学上使用逗号作为千位分隔符。

  ✓例如：

```
>>> 1_000_000
1000000
>>> 1_2_3_4
1234
>>> 1_2 + 3_4j
(12+34j)
>>> 1_2.3_45
12.345
```



1.3 基础知识---Python的字符串类型

1.3.4 Python的字符串类型

- 用单引号、双引号或三引号界定的符号系列称为字符串
- 单引号、双引号、三单引号、三双引号可以互相嵌套，用来表示复杂字符串
  - 例如： `'abc'、'123'、'中国'、"Python"、'''Tom said, "Let's go"'''`
- **字符串属于不可变序列**
- 空字符串表示为''或 ""
- 三引号'''或"""表示的字符串可以换行，支持排版较为复杂的字符串；三引号还可以在程序中表示较长的注释。

1.3 基础知识---Python的字符串类型

- 字符串之间可以通过+或*进行连接
  - 加法操作(+)将两个字符串连接成为一个新的字符串
  - 乘法操作(*)生成一个由其本身字符串重复连接而成的字符串
- len()函数能否返回一个字符串的长度
  - ✓ 例如：

```
>>> a = 'abc' + '123'          #生成新字符串
>>> x = '1234''abcd'
>>> x
'1234abcd'
>>> x = x + ',.;'
>>> x
'1234abcd,.;'
>>> x = x'efg'                 #不允许这样连接字符串
SyntaxError: invalid syntax
```

1.3 基础知识---Python的字符串类型

常用转义字符

转义字符	含义	转义字符	含义
\b	退格，把光标移动到前一列位置	\\	一个斜线\
\f	换页符	\’	单引号’
\n	换行符	\”	双引号”
\r	回车	\ooo	3位八进制数对应的字符
\t	水平制表符	\xhh	2位十六进制数对应的字符
\v	垂直制表符	\uhhhh	4位十六进制数表示的Unicode字符

✓ 例如：

```
>>> print('Hello\nWorld')                    #包含转义字符的字符串
Hello
World
```



1.3 基础知识---Python的字符串类型

- 字符串界定符前面加字母r或R表示原始字符串，其中的特殊字符不进行转义，但字符串的最后一个字符不能是\。
  - 原始字符串主要用于正则表达式、文件路径或者URL的场合。
  - ✓ 例如：

```
>>> path = 'C:\Windows\notepad.exe'
>>> print(path)                              #字符\n被转义为换行符
C:\Windows
otepad.exe
>>> path = r'C:\Windows\notepad.exe' #原始字符串，任何字符都不转义
>>> print(path)
C:\Windows\notepad.exe
```

1.3 基础知识---运算符和表达式

1.3.5 运算符和表达式

运算符	功能说明
+	算术加法，列表、元组、字符串合并与连接，正号
-	算术减法，集合差集，相反数
*	算术乘法，序列重复
/	真除法　　6/5=1.2　　6//5=1　　6.0//5=1.0
//	求整商，但如果操作数中有实数的话，结果为实数形式的整数
%	求余数，字符串格式化
**	幂运算
<、<=、>、>=、==、!=	（值）大小比较，集合的包含关系比较
or，and，not	逻辑或、逻辑与、逻辑非
in	成员测试
is	对象同一性测试，即测试是否为同一个对象或内存地址是否相同
|、^、&、<<、>>、~	位或、位异或、位与、左移位、右移位、位求反
&、|、^，-	集合交集、并集、对称差集，差集

1.3.5 运算符和表达式

- +运算符除了用于算术加法外，还可用于列表、元组、字符串的连接，但不支持不同类型的对象之间相加或连接。
- 例如：

```
>>> 'abcd' + '1234'
'abcd1234'
>>> 'A' + 1
TypeError: Can't convert 'int' object to
str implicitly
>>> True + 3                    #Python内部
把True当作1处理
4
>>> False + 3                   #把False当作0
处理
3
```



1.3 基础知识---运算符和表达式

- *运算符不仅可以用于数值乘法，还可以用于列表、字符串、元组等类型
- 当列表、字符串或元组等类型变量与整数进行“*”运算时，表示对内容进行重复并返回重复后的新对象。

✓ 例如：

```
>>> 2.0 * 3                    #浮点数与整数相乘
6.0
>>> (3+4j) * 2                 #复数与整数相乘
(6+8j)
>>> (3+4j) * (3-4j)            #复数与复数相乘
(25+0j)
>>> "a" * 10                   #字符串重复
'aaaaaaaaaa'
```

1.3 基础知识---运算符和表达式

- Python中的除法有两种，“/”和“//”分别表示除法和整除运算(//向下取接近商的整数)。
  - ✓ 例如：

```
>>> 3 / 5
0.6
>>> 3 // 5
0
>>> 3.0 / 5
0.6
>>> 3.0 // 5
0.0
>>> 9 // 2
4
>>> -9 // 2
-5
```



1.3 基础知识---运算符和表达式

DMIC -HFUT-

- %运算符除去可用于字符串格式化之外，也可对整数和浮点数计算余数。
  - 由于浮点数的精确度影响，计算结果可能略有误差。
  - ✓ 例如：

```
>>> 3.1 % 2
1.1
>>> 6.3 % 2.1
2.0999999999999996
>>> 6 % 2
0
>>> -17 % 4
3
>>> 17 % -4
-3
>>> 5.7 % 4.8
0.9000000000000004
```

#余数与%右侧的运算数符号一致

#(17-(-3))能被(-4)整除



1.3 基础知识---运算符和表达式

- 关系运算符可以连用，一般用于同类型对象之间值的大小比较，或者测试集合之间的包含关系
  - ✓例如：

```
>>> 1 < 3 < 5                          #等价于1 < 3 and 3 < 5
True
>>> 'Hello' > 'world'                  #比较字符串大小
False
>>> 'Hello' > 3                        #字符串和数字不能比较
TypeError: unorderable types: str() > int()
```



1.3 基础知识---运算符和表达式

DMiC -HFUT-

- 成员测试运算符in用于成员测试
  - 即测试一个对象是否为另一个对象的元素。

✓ 例如：

```
>>> 5 in range(1, 10, 1) #range()是用来生成指定范围数字的内置函数
True
>>> 'abc' in 'abcdefg'    #子字符串测试
True
>>> for i in (3, 5, 7):   #循环，成员遍历
    print(i, end='\t')

3   5   7
```

1.3 基础知识---运算符和表达式

is 与 == 区别:

is 用于判断两个变量引用对象是否为同一个， == 用于判断引用变量的值是否相等。

```
>>> a=[1,2,3]
>>> b=a
>>> b is a
True
>>> a=[1,2,3]
>>> b=[1,2,3]
>>> b is a
False
>>> b=a[:]
>>> b is a
False
>>> b==a
True
>>>
```

1.3 基础知识---运算符和表达式

- **位运算符只能用于整数**，其内部执行过程为：首先将整数转换为二进制数，然后右对齐，必要的时候左侧补0，按位进行运算，最后再把计算结果转换为十进制数字返回

✓ **例如：**

```
>>> 3 << 2        #把3左移2位
12
>>> 3 & 7         #位与运算
3
>>> 3 | 8         #位或运算
11
>>> 3 ^ 5         #位异或运算
6
```

位运算符规则

位与运算： 1&1=1、1&0=0&1=0&0=0

位或运算： 1|1=1|0=0|1=1、0|0=0

位异或运算： 1^1=0^0=0、1^0=0^1=1

左移位时右侧补0，每左移一位相当于乘以2

移位时左侧补0，每右移一位相当于整除以2



1.3 基础知识---运算符和表达式

- 集合的交集、并集、对称差集等运算借助于位运算符来实现
- 而差集则使用减号运算符实现
- ✓ 例如：

```
>>> {1, 2, 3} | {3, 4, 5}                #并集，自动去除重复元素
{1, 2, 3, 4, 5}
>>> {1, 2, 3} & {3, 4, 5}                #交集
{3}
>>> {1, 2, 3} ^ {3, 4, 5}                #对称差集
{1, 2, 4, 5}
>>> {1, 2, 3} - {3, 4, 5}                #差集
{1, 2}
```

1.3 基础知识---运算符和表达式

- and和or具有惰性求值特点，只计算必须计算的表达式

✓ 例如：

```
>>> 3>5 and a>3          #注意，此时并没有定义变量a
False
>>> 3>5 or a>3           #3>5的值为False，所以需要计算后面表达式
NameError: name 'a' is not defined
>>> 3<5 or a>3           #3<5的值为True，不需要计算后面表达式
True
>>> 3 and 5
5
>>> 3 and 5>2
True
```

在python中and与or执行布尔逻辑运算，但返回的是实际值。

1.全为and，如果都为真，则返回最后一个变量值；如果为假，则返回第一个假值

2.全为or，如果都为假则返回最后一个值；如果为真，则返回第一个真值

#最后一个计算的表达式的值作为整个表达式的值

- Python中单个任何类型的对象或常数属于合法表达式，使用运算符连接的变量和常量以及函数调用的任意组合也属于合法的表达式

✓ 例如：

```
>>> 'Hello' + ' ' + 'world'
'Hello world'
```



1.3 基础知识---运算符和表达式

DMIC -HFUT-

➢ 特殊说明

- 逗号并不是运算符，只是一个普通分隔符
  - ✓ 例如：

```
>>> x = 3, 5
>>> x
(3, 5)
>>> 3==3, 5
(True, 5)
```

- Python不支持++和--运算符，虽然在形式上有时似乎可这样用，但实际上是另外的含义
  - ✓ 例如：

```
>>> i = 3
>>> ++i
3                                   #正正得正
>>> i++                             #Python不支持++运算符，语法错误
SyntaxError: invalid syntax
>>> --i                             #负负得正
3
>>> -(-i)                           #与--i等价
3
```

1.3.6 常用内置函数

- 内置函数不需要导入任何模块即可使用
- 执行下面的命令可以列出所有内置函数

```
>>> dir (__builtins__)
```

函数	功能简要说明
abs(x)	返回数字x的绝对值或复数x的模
all(iterable)	如果对于可迭代对象中所有元素x都等价于True，也就是对于所有元素x都有bool(x)等于True，则返回True。对于空的可迭代对象也返回True
any(iterable)	只要可迭代对象iterable中存在元素x使得bool(x)为True，则返回True。对于空的可迭代对象，返回False
ascii(obj)	把对象转换为ASCII码表示形式，必要的时候使用转义字符来表示特定的字符
bin(x)	把整数x转换为二进制串表示形式
bool(x)	返回与x等价的布尔值True或False
bytes(x)	生成字节串，或把指定对象x转换为字节串表示形式
callable(obj)	测试对象obj是否可调用。类和函数是可调用的，包含__call__()方法的类的对象也是可调用的
compile()	用于把Python代码编译成可被exec()或eval()函数执行的代码对象
complex(real, [imag])	返回复数
chr(x)	返回Unicode编码为x的字符

1.3.6 常用内置函数

函数	功能简要说明
delattr(obj, name)	删除属性，等价于del obj.name
dir(obj)	返回指定对象或模块obj的成员列表，如果不带参数则返回当前作用域内所有标识符
divmod(x, y)	返回包含整商和余数的元组((x-x%y)/y, x%y)
enumerate(iterable[, start])	返回包含元素形式为(0, iterable[0]), (1, iterable[1]), (2, iterable[2]), ...的迭代器对象
eval(s[, globals[, locals]])	计算并返回字符串s中表达式的值
exec(x)	执行代码或代码对象x
exit()	退出当前解释器环境
filter(func, seq)	返回filter对象，其中包含序列seq中使得单参数函数func返回值为True的那些元素，如果函数func为None则返回包含seq中等价于True的元素的filter对象
float(x)	把整数或字符串x转换为浮点数并返回
frozenset([x]))	创建不可变的集合对象
getattr(obj, name[, default]	获取对象中指定属性的值，等价于obj.name，如果不存在指定属性则返回default的值，如果要访问的属性不存在并且没有指定default则抛出异常



1.3 基础知识---常用内置函数

DMiC -HFUT-

1.3.6 常用内置函数

函数	功能简要说明
globals()	返回包含当前作用域内全局变量及其值的字典
hasattr(obj, name)	测试对象obj是否具有名为name的成员
hash(x)	返回对象x的哈希值，如果x不可哈希则抛出异常
help(obj)	返回对象obj的帮助信息
hex(x)	把整数x转换为十六进制串
id(obj)	返回对象obj的标识（内存地址）
input([提示])	显示提示，接收键盘输入的内容，返回字符串
int(x[, d])	返回实数（float）、分数（Fraction）或高精度实数（Decimal）x的整数部分，或把d进制的字符串x转换为十进制并返回，d默认为十进制
isinstance(obj, class-or-type-or-tuple)	测试对象obj是否属于指定类型（如果有多个类型的话需要放到元组中）的实例
iter(...)	返回指定对象的可迭代对象
len(obj)	返回对象obj包含的元素个数，适用于列表、元组、集合、字典、字符串以及range对象和其他可迭代对象

1.3 基础知识---常用内置函数

函数	功能简要说明
list([x])、set([x])、tuple([x])、dict([x])	把对象x转换为列表、集合、元组或字典并返回，或生成空列表、空集合、空元组、空字典
locals()	返回包含当前作用域内局部变量及其值的字典
map(func, *iterables)	返回包含若干函数值的map对象，函数func的参数分别来自于iterables指定的每个迭代对象，
max(x)、 min(x)	返回可迭代对象x中的最大值、最小值，要求x中的所有元素之间可比较大小，允许指定排序规则和x为空时返回的默认值
next(iterator[, default])	返回可迭代对象x中的下一个元素，允许指定迭代结束之后继续迭代时返回的默认值
oct(x)	把整数x转换为八进制串
open(name[, mode])	以指定模式mode打开文件name并返回文件对象
ord(x)	返回1个字符x的Unicode编码
pow(x, y, z=None)	返回x的y次方，等价于x ** y或(x ** y) % z
print(value, ..., sep=' ', end='\n', file = sys.stdout, flush=False)	基本输出函数
quit()	退出当前解释器环境
range([start,] end [, step])	返回range对象，其中包含左闭右开区间[start,end)内以step为步长的整数

1.3.6 常用内置函数

函数	功能简要说明
repr(obj)	返回对象obj的规范化字符串表示形式，对于大多数对象有eval(repr(obj))==obj
reversed(seq)	返回seq（可以是列表、元组、字符串、range以及其他可迭代对象）中所有元素逆序后的迭代器对象
round(x [, 小数位数])	对x进行四舍五入，若不指定小数位数，则返回整数
sorted(iterable, key=None, reverse=False)	返回排序后的列表，其中iterable表示要排序的序列或迭代对象，key用来指定排序规则或依据，reverse用来指定升序或降序。该函数不改变iterable内任何元素的顺序
str(obj)	把对象obj直接转换为字符串
sum(x, start=0)	返回序列x中所有元素之和，返回start+sum(x)
type(obj)	返回对象obj的类型
zip(seq1 [, seq2 [...]])	返回zip对象，其中元素为(seq1[i], seq2[i], ...)形式的元组，最终结果中包含的元素个数取决于所有参数序列或可迭代对象中最短的那个

1.3.6 常用内置函数-具体示例

- 输入函数：input( )
  - 可输入数字、字符串和其它任意类型对象，返回结果都是字符串。
  - ✓ 例如：>>> x = input('提示：')
- 输出函数：print()函数进行输出。

```
>>> print(3, 5, 7)
3 5 7
>>> print(3, 5, 7, sep=',')      #指定分隔符
3,5,7
>>> for i in range(10,20):
        print(i, end=' ')          #不换行

10 11 12 13 14 15 16 17 18 19
```

- 在Python 3.x中则需要使用下面的方法进行重定向：

```
>>> fp = open(r'D:\test.txt', 'a+')
>>> print('Hello,world!', file = fp)
>>> fp.close()
```

```
>>> with open(r'D:\mytest.txt', 'a+') as fp:
        print('Hello,world!', file=fp)
```

1.3.6 常用内置函数-具体示例

- bin()、oct()、hex()用来将整数转换为二进制、八进制和十六进制形式，这三个函数都要求参数必须为整数。

✓ 例如：

```
>>> bin(32)                    # 把数字转换为二进制串
'0b100000'
>>> oct(32)                    # 转换为八进制串
'0o40'
>>> hex(555)                   # 转换为十六进制串
'0x20'
```

- int()用来把实数转换为整数，或把数字字符串按指定进制转换为十进制数。

✓例如：

```
>>> int(-3.5)
-3
>>> int('101', 2)              # 二进制
5
```

1.3 基础知识---常用内置函数

- eval(<字符串>)函数能够以Python表达式的方式解析并执行字符串，将返回结果输出

```
>>>x = 1
>>>eval("x + 1")
2
>>>eval("1.1 + 2.2")
3.3
```

- range()语法格式为range([start,] end [, step])，返回具有惰性求值特点的range对象，其中包含左闭右开区间[start,end)内以step为步长的整数。

```
>>> range(5)                    #start默认为0，step默认为1
range(0, 5)
>>> list(_)
[0, 1, 2, 3, 4]
>>> list(range(1, 10, 2))       #指定起始值和步长
[1, 3, 5, 7, 9]
>>> list(range(9, 0, -2))       #步长为负数时，start应比end大
[9, 7, 5, 3, 1]
```

1.3 基础知识---常用内置函数

- ord()和chr()是一对功能相反的函数：ord()用来返回单个字符的序数或Unicode码，而chr()则用来返回某序数对应的字符
- str()则直接将其任意类型参数转换为字符串。

```
>>> ord('a')
97
>>> chr(ord('A')+1)
'B'
```

```
>>> chr(65)
'A'
>>> str(12)
'12'
```

- max()、min()、sum()这三个内置函数分别用于计算列表、元组或其他可迭代对象中所有元素最大值、最小值以及所有元素之和

```
>>> import random
>>> a = [random.randint(1,100) for i in range(10)]
                    #列表推导式
>>> a
  [72, 26, 80, 65, 34, 86, 19, 74, 52, 40]
>>> print(max(a), min(a), sum(a))
  86 19 548
>>> sum(a)/len(a)                    #计算该列表中的所有元素的平均值
  54.8
```

1.3.6 常用内置函数-具体示例

- 内置函数type()和isinstance()可以判断数据类型。
  - 例如：

```
>>> type([3])
<class 'list'>
>>> type({3}) in (list, tuple, dict) #判断{3}是否为list,tuple或dict类型的实例
False
>>> isinstance(3, int)
True
>>> isinstance(3j, (int, float, complex))
True
```

- sorted()对列表、元组、字典、集合或其他可迭代对象进行排序并返回新列表。
  - 例如：

```
>>> x = ['aaaa', 'bc', 'd', 'b', 'ba']
>>> sorted(x, key=lambda item: (len(item), item))
      #先按长度排序，长度一样的正常排序

['b', 'd', 'ba', 'bc', 'aaaa']
```

1.3 基础知识---常用内置函数

- zip()函数用来把多个可迭代对象中的元素压缩到一起，返回一个可迭代的zip对象，其中每个元素都是包含原来的多个可迭代对象对应位置上元素的元组，如同拉拉链一样。



✓ 例如：

```
>>> x = zip('abcd', '1234')
>>> list(x)
[('a', '1'), ('b', '2'), ('c', '3'), ('d', '4')]
```

- 内置函数filter()将一个单参数函数作用到一个序列上，返回该序列中使得该函数返回值为True的那些元素组成的filter对象，如果指定函数为None，则返回序列中等价于True的元素。

✓ 例如：

```
>>> seq = ['foo', 'x41', '?!', '***']
>>> list(filter(str.isalnum, seq))
['foo', 'x41']
```

注： isalnum() 方法检测字符串是否由字母和数字组成。

1.3 基础知识---常用内置函数

- enumerate() 函数用于将一个可遍历的数据对象(如列表、元组或字符串)组合为一个索引序列，同时列出数据和数据下标，一般用在 for 循环当中。

```
>>> for index, value in enumerate(range(10, 15)):  #枚举range对象中的元素
    print((index, value), end=' ')
(0, 10) (1, 11) (2, 12) (3, 13) (4, 14)
>>> list(enumerate('abcd'))                        #枚举字符串中的元素
[(0, 'a'), (1, 'b'), (2, 'c'), (3, 'd')]
```

- 内置函数map()把一个函数func依次映射到序列或迭代器对象的每个元素上，并返回一个可迭代的map对象作为结果，map对象中每个元素是原序列中元素经过函数func处理后的结果。

✓ 例如：

```
>>> list(map(str, range(5)))
['0', '1', '2', '3', '4']
>>> def add5(v):
    return v+5
>>> list(map(add5, range(10)))   #把单参数函数映射到一个序列的所有元素
[5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
```

1.3 基础知识---常用内置函数

- map、filter、enumerate、zip等对象不仅具有惰性求值的特点，还有另外一个特点：访问过的元素不可再次访问。

```
>>> x = map(str, range(10)) #Python 2.x 返回列表。Python 3.x 返回迭代器。

>>> list(x)
['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
>>> list(x)
[]
>>> x = map(str, range(10))
>>> '2' in x
True
>>> '2' in x
False
```

- **reversed()对可迭代对象（生成器对象和具有惰性求值特性的zip、map、filter、enumerate等类似对象除外）进行翻转（首尾交换）并返回可迭代的reversed对象。**

```
>>> x = ['aaaa', 'bc', 'd', 'b', 'ba']
>>> reversed(x)                    #逆序，返回reversed对象
<list_reverseiterator object at 0x0000000002E6C3C8>
>>> list(reversed(x))
['ba', 'b', 'd', 'bc', 'aaaa']
```

1.3 基础知识---对象的删除

1.3.7 对象的删除

- Python具有自动内存管理功能，Python解释器会跟踪所有的值，一旦发现某个值不再有任何变量指向，将会自动删除该值。
- 显式释放自己申请的资源是程序员的好习惯之一，也是程序员素养的重要体现之一。
- del命令：显式删除对象并解除与值之间的指向关系。
  - ✓ 例如：

```
>>> y = 3
>>> z = y
>>> print(y)
3
>>> del y                    #删除对象
>>> print(y)
NameError: name 'y' is not defined
>>> print(z)
3
>>> del z
>>> print(z)
NameError: name 'z' is not defined
```

- del命令无法删除元组或字符串中的元素，只可以删除整个元组或字符串
  - ✓ 因为这两者均属于不可变序列。
  - ✓ 例如：

```
>>> s = "hello"
>>> del s[0]
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    del s[0]
TypeError: 'str' object doesn't support item
deletion
```

1.3.8 模型的导入与使用

- Python默认安装仅包含部分基本或核心模块，用户可安装大量的扩展模块，pip是管理模块的重要工具。
- 在Python启动时，仅加载了很少的一部分模块，在需要时由程序员显式地加载（可能需要先安装）其他模块。
- **减小运行的压力，仅加载真正需要的模块和功能，且具有很强的可扩展性。**
- 可以使用sys.modules.items()显示所有预加载模块的相关信息。
- 扩展模块的导入方法
  - import 模块名
  - from 模块名 import 对象名[as 别名]
  - from math import *
  - ✓

```
>>> from math import sin
>>> sin(3)
0.1411200080598672
>>> from math import sin as f
>>> f(3)
0.141120008059867
```

1.3.9 Python文件

- **.py：Python源文件，由Python解释器负责解释执行。**
- **.pyw：Python源文件，常用于图形界面程序文件。**
- **.pyc：Python字节码文件，无法使用文本编辑器直接查看其内容，可用于隐藏Python源代码和提高运行速度。**
  - ✓ 对于Python模块，第一次被导入时将被编译成字节码的形式，并在以后再次导入时优先使用“.pyc”文件，以提高模块的加载和运行速度。
  - ✓ 对于非模块文件，直接执行时并不生成“.pyc”文件，但可使用py_compile模块的compile()函数进行编译以提高加载和运行速度。
  - ✓ Python还提供了compileall模块，其中包含compile_dir()、compile_file()和compile_path()等方法，用来支持批量Python源程序文件的编译。
- **.pyd：一般是由其他语言编写并编译的二进制文件，常用于实现某些软件工具的Python编程接口插件或Python动态链接库。**



1.3 基础知识-Python脚本的“__name__”属性

1.3.9 Python脚本的“__name__”属性

- 每个Python脚本在运行时都有一个“__name__”属性。
  - 如果脚本作为模块被导入，则其“__name__”属性值被自动设置为模块名；
  - 如果脚本独立运行，则其“__name__”属性值被自动设置为“__main__”。

✓ 例如：假设文件nametest.py中只包含下面一行代码：

```
print(__name__)
```

- 在IDLE中直接运行该程序时，或者在命令行提示符环境中运行该程序文件时，运行结果如下：

```
__main__
```

- 而将该文件作为模块导入时得到如下执行结果：

```
>>> import nametest
nametest
```

1.3.9 Python脚本的“__name__”属性

- 利用“__name__”属性即可控制Python程序的运行方式。

✓ 例如，编写一个包含大量可被其他程序利用的函数的模块，而不希望该模块可以直接运行，则可以在程序文件中添加以下代码：

```
if __name__ == '__main__':
    print('Please use me as a module.')
```

- 程序直接执行时将会得到提示“Please use me as a module.”，而使用import语句将其作为模块导入后可以使用其中的类、方法、常量或其他成员。

1.4 Python代码规范

- (1) 缩进
  - 缩进指每一行代码开始前的空白区域，用来表示代码之间的包含和层次关系；　1个缩进 = 4个空格
  - 类定义、函数定义、选择结构、循环结构、with块，行尾的冒号表示缩进的开始。
  - python程序是依靠代码块的缩进来体现代码之间的逻辑关系的，缩进结束就表示一个代码块结束了。

```
TempStr = input("请输入带有符号的温度值: ")
if TempStr[-1] in ['F','f']:
    C = (eval(TempStr[0:-1]) - 32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C','c']:
    F = 1.8*eval(TempStr[0:-1]) + 32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```

行尾的冒号和下一行的缩进，表示下一个代码块的开始，而缩进的结束则表示此代码块的结束。

1.4 Python代码规范

- （2）注释
  - 以#开始，表示本行#之后的内容为注释。
  - 包含在一对三引号'''...'''或"""..."""之间且不属于任何语句的内容将被解释器认为是注释。

```
File  Edit  Format  Run  Options  Window  Help
s = 3+4          #这是一个加法程序
print(s)
'''
这是一个简单的
示例程序
'''
```

1.4 Python代码规范

- （3）语句过长的处理方法
  - 可在行尾加上续行符\来换行分成多行；
  - 可使用括号来包含多行内容。

```
x = 1 + 2 + 3\
    + 4 + 5\
    + 6

y = (1 + 2 + 3
     + 4 + 5
     + 6)
```

- （4）必要的空格与空行
  - 运算符两侧、逗号后面建议增加一个空格。
    - ✓ >>>x = 1, 2
  - 不同功能的代码块之间、不同的函数定义之间建议增加一个空行以增加可读性。

使用计算机解决问题

- 分析问题，分析问题的计算部分
- 划分边界，划分问题的功能边界
- 设计算法，设计问题的求解算法
- 编写程序，编写问题的计算程序
- 调试测试，调试和测试程序
- 升级维护，适应问题的升级维护

◆程序的基本编写方法

■IPO程序编写方法

- **输入（Input）数据：**文件输入、控制台输入、交互界面输出、随机数据输入等
- **处理（Process）数据：**处理是程序对输入数据进行计算产生输出结果的过程。计算问题的处理方法统称为“算法”
- **输出（Output）数据：**控制台输出、图形输出、文件输出、网络输出、操作系统内部变量输出等。

1.5 Python快速入门

例1-1：用户输入一个三位自然数，计算并输出其百位、十位和个位上的数字。

解答1：

```
x = input('请输入一个三位数：')
x = int(x)
a = x // 100     # "//"表示整数除法，返回不大于结果的最大的整数。
b = x // 10 % 10
c = x % 10
print(a, b, c)
```

不限位数怎么办？

解答2：

```
x = input('请输入一个三位数：')
x = int(x)
a, b = divmod(x, 100)   #divmod() 函数返回一个包含商和余数的元组
b, c = divmod(b, 10)
print(a, b, c)
```

解答3：

```
x = input('请输入一个三位数：')
a, b, c = map(int, x)     #以参数序列中的每一个元素调用 int 函数
print(a, b, c)
```

用户输入一个整数，如何确定位数？

1）利用str()函数将数字转化成字符串，再利用len()函数判断位长。

2）或者 利用除10取商，通过循环次数判断位数。

```
>>> a=int(input("please input number:"))
please input number:3456789
>>> k=len(str(a))
>>> k
7
```

```
k=0
a=int(input("please input number:"))
while a!=0:
    a=int(a/10)
    print(a)
    k+=1
print(k)
```

例1-2：已知三角形的两边长及其夹角，求第三边长。

解答1：

```
import math
x = input('输入两边长及夹角（度）：')
a, b, theta = map(float, x.split())
c = math.sqrt(a**2 + b**2 - 2*a*b*math.cos(theta*math.pi/180))
print('c=', c)
```

外部扩展模块

```
输入两边长及夹角（度）：3 4 90
c= 5.0
>>>
```

例1-3：任意输入三个英文单词，按字典顺序输出。

解答1：

```
s = input('x, y, z=')
x, y, z = s.split(',')
if x > y:
    x, y = y, x
if x > z:
    x, z = z, x
if y > z:
    y, z = z, y
print(x, y, z)
```

有没有更简单的方法？

解答2：

```
s = input('x, y, z=')
x, y, z = sorted(s.split(','))  #split返回一个包含分割结果的列表
print(x, y, z)
```

1.6 本章小结

内容回顾

- Python基础知识
  - 变量、数字类型、字符串类型、常用内置函数等
- Python代码规范
- Python案例入门

思考

- 如何编写自己的Python代码？